THE UNIVERSITY OF ALABAMA®

WHERE LEGENDS ARE MADE®

Binary Trees

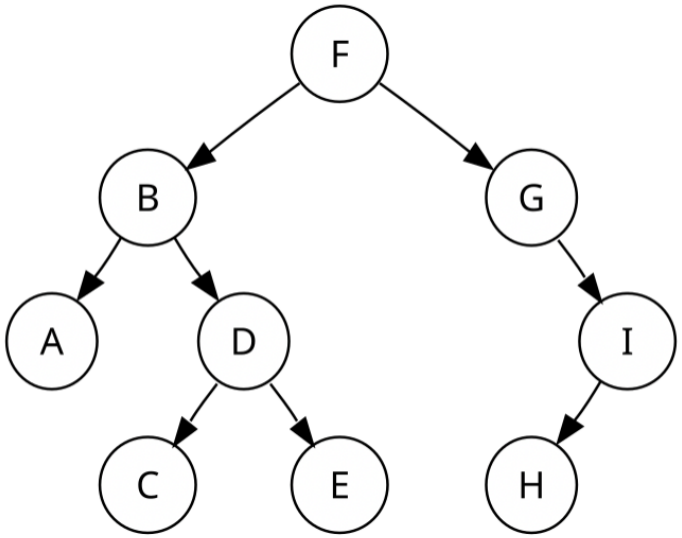
And their Applications with Stacks

CS 201: Data Structures and Algorithms

Jiacheng Li, Ph.D.

Binary Tree

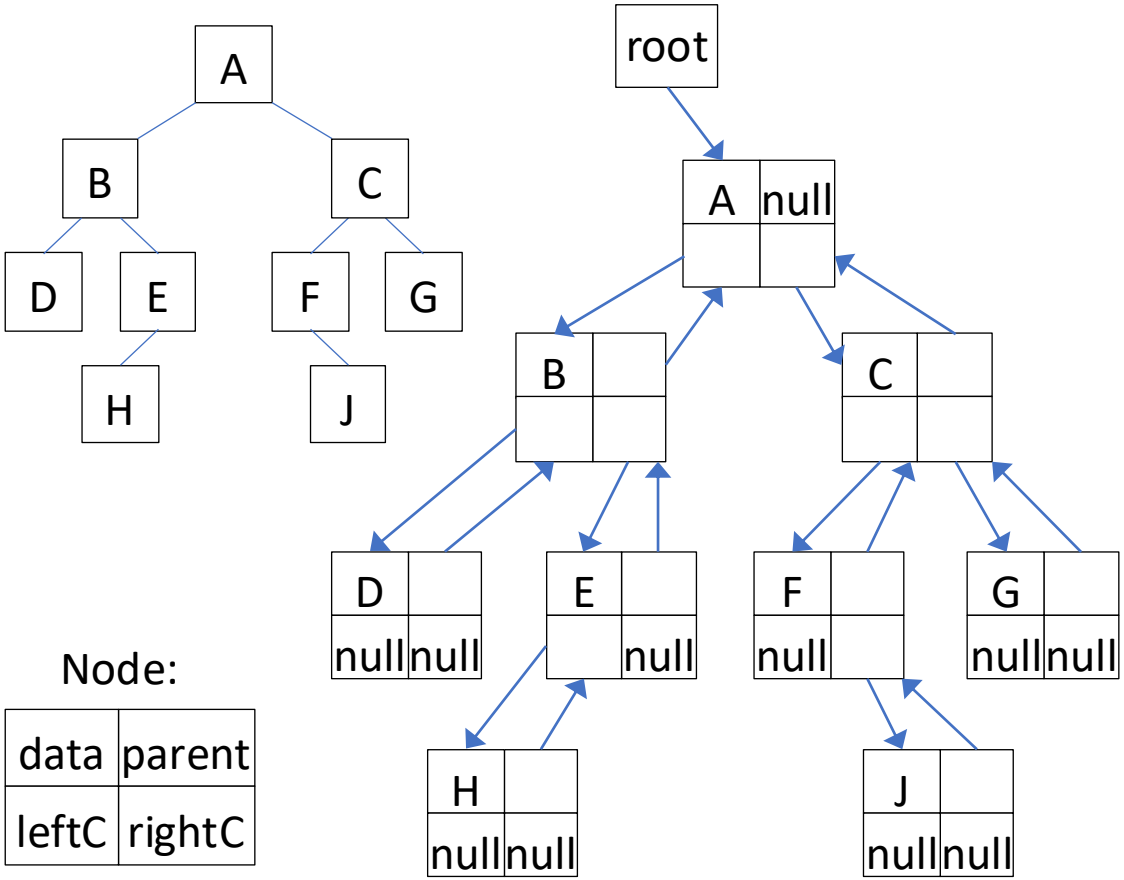
- A *binary tree* is a type of data structure used in computer science to organize and store data in a hierarchical way.



Binary Tree Programmatically

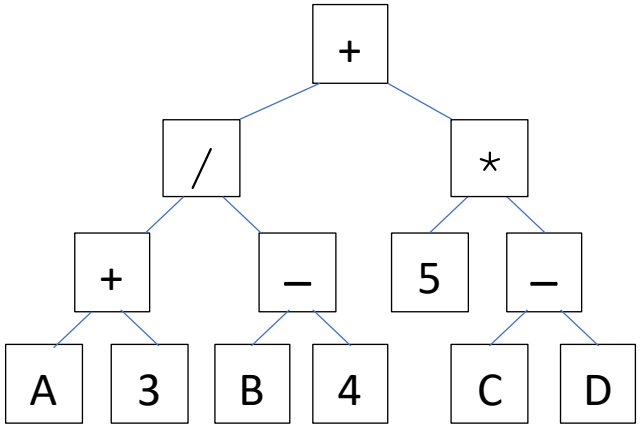
```
class Node {
  ElementType data;
  Node *parent, *leftChild, *rightChild;
}

Class BinaryTree {
  Node *root;
}
```

Binary Tree Visually



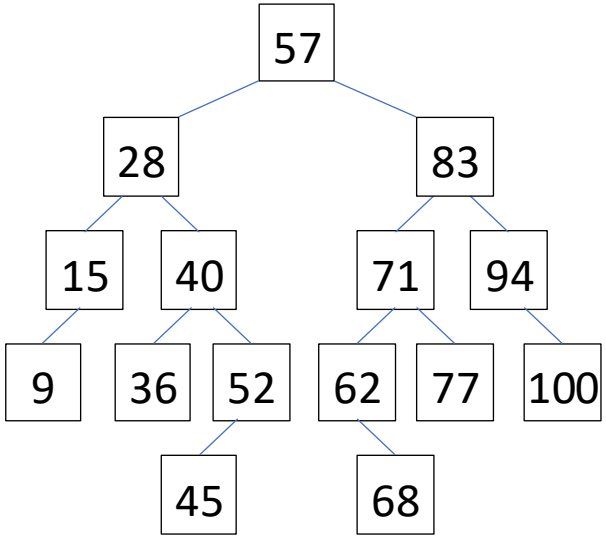
Applications

- Arithmetic Expression Trees
  - Example: (A+3)/(B-4) + 5*(C-D)



Applications (cont.)

- Binary Search Trees
  - (More about these later in the course)
  - If node X is in left subtree of node Y,
    - then X.data ≤ Y.data
  - If node Z is in right subtree of node Y,
    - then Z.data ≥ Y.data



Binary Tree Traversals

Depth-First:

- Preorder
- Inorder
- Postorder

Breadth-First:

- Level-order

Binary Tree Traversals

Depth-First:

- Preorder
- Inorder
- Postorder

Breadth-First:

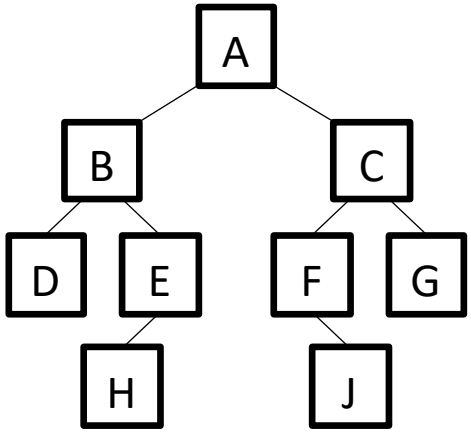
- Level-order

Preorder Traversal (CLR)

```
void preorder(BinaryTree T) {
  preorder(T->root);          // A wrapper function
}

void preorder(Node *p) {
  if (p == nullptr) return;
  visit(p); // Example: print(p->data); // Current
  preorder(p->leftChild); // Left
  preorder(p->rightChild); // Right
}
```

Preorder Traversal

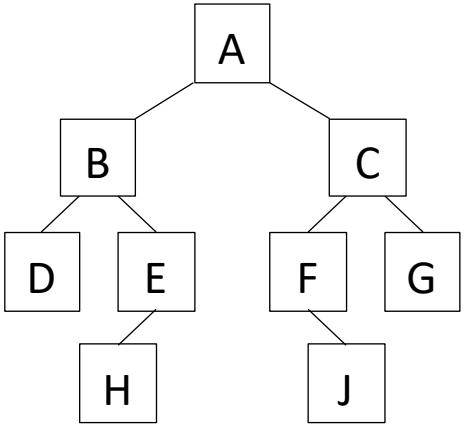
```
void preorder(Node *p) {
  if (p == nullptr) return;
  visit(p); // Example: print(p->data);
  preorder(p->leftChild);
  preorder(p->rightChild);
}
```



Prefix Notation: A B D E H C F J G

Preorder Traversal

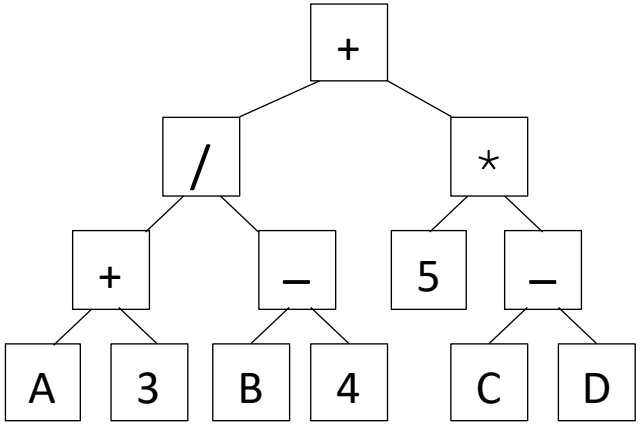
```
void preorder(Node *p) {
  if (p == nullptr) return;
  visit(p); // Example: print(p->data);
  preorder(p->leftChild);
  preorder(p->rightChild);
}
```



Prefix Notation: A <u>B D E H</u> <u>C F J G</u>

Preorder Traversal

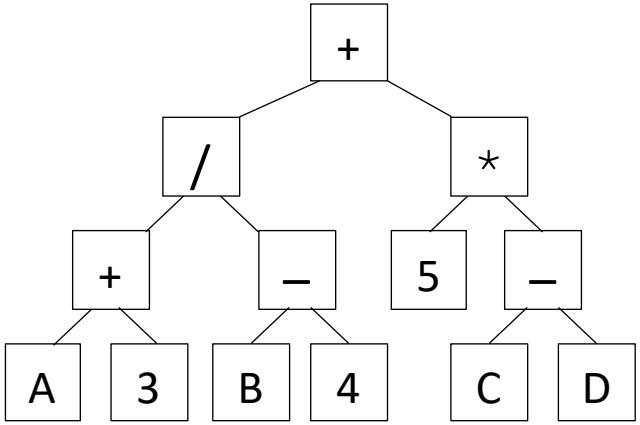
```
void preorder(Node *p) {
  if (p == nullptr) return;
  visit(p); // Example: print(p->data);
  preorder(p->leftChild);
  preorder(p->rightChild);
}
```



Prefix: try it

Preorder Traversal

```
void preorder(Node *p) {
  if (p == nullptr) return;
  visit(p); // Example: print(p->data);
  preorder(p->leftChild);
  preorder(p->rightChild);
}
```



Prefix: + <u>/ + A 3 – B 4</u> <u>* 5 – C D</u>

Binary Tree Traversals

Depth-First:

- Preorder
- Postorder
- Inorder

Breadth-First:

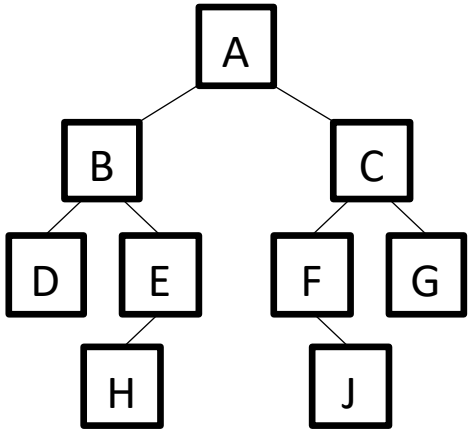
- Level-order

Postorder Traversal (LRC)

```
void postorder(BinaryTree T) {
  postorder(T->root);
}

void postorder(Node *p) {
  if (p == nullptr) return;
  postorder(p->leftChild); // Left
  postorder(p->rightChild); // Right
  visit(p); // Current
}
```

Postorder Traversal

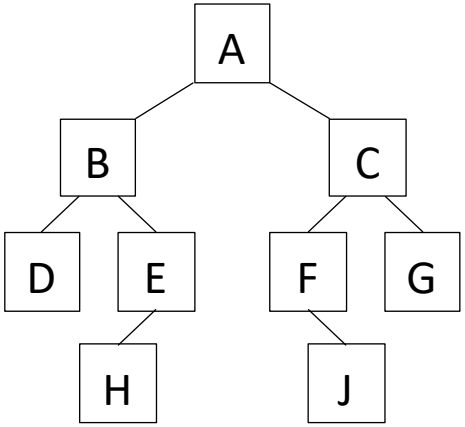
```
void postorder(Node *p) {
  if (p == nullptr) return;
  postorder(p->leftChild);
  postorder(p->rightChild);
  visit(p);
}
```



Postfix Notation: D H E B J F G C A

Postorder Traversal

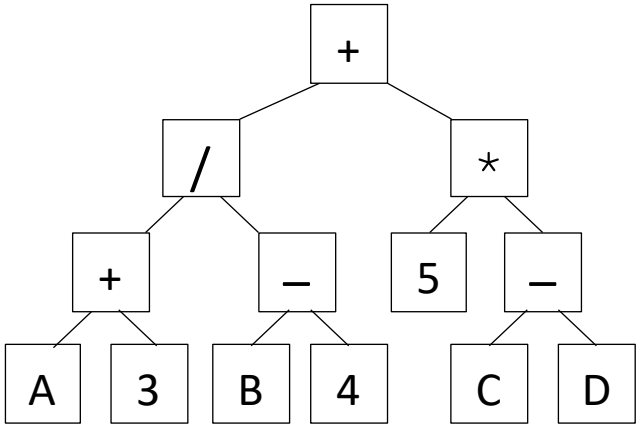
```
void postorder(Node *p) {
  if (p == nullptr) return;
  postorder(p->leftChild);
  postorder(p->rightChild);
  visit(p);
}
```



Postfix Notation: <u>D H E B</u> <u>J F G C</u> A

Postorder Traversal

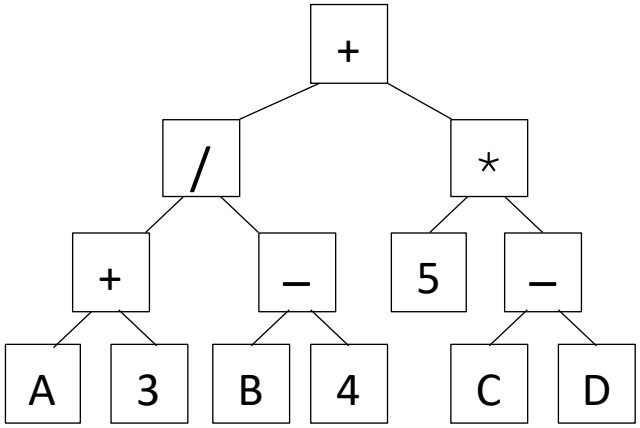
```
void postorder(Node *p) {
  if (p == nullptr) return;
  postorder(p->leftChild);
  postorder(p->rightChild);
  visit(p);
}
```



Postfix: try it

Postorder Traversal

```
void postorder(Node *p) {
  if (p == nullptr) return;
  postorder(p->leftChild);
  postorder(p->rightChild);
  visit(p);
}
```



Postfix: <u>A 3 + B 4 – /</u> <u>5 C D – *</u> +

Binary Tree Traversals

Depth-First:

- Preorder
- Postorder
- Inorder

Breadth-First:

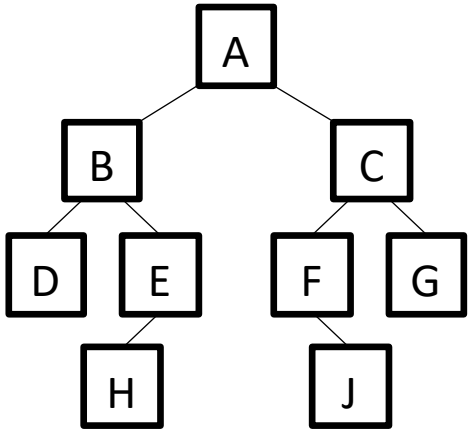
- Level-order

Inorder Traversal (LCR)

```
void inorder(BinaryTree T) {
  inorder(T->root);
}

void inorder(Node *p) {
  if (p == nullptr) return;
  inorder(p->leftChild); // Left
  visit(p); // Current
  inorder(p->rightChild); // Right
}
```

Inorder Traversal

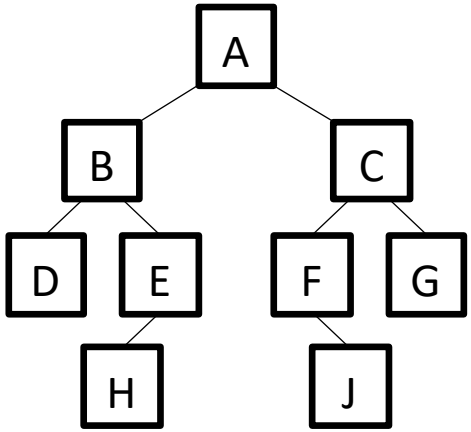
```
void inorder(Node *p) {
  if (p == nullptr) return;
  inorder(p->leftChild);
  visit(p);
  inorder(p->rightChild);
}
```



Infix Notation: D B H E A F J C G

Inorder Traversal

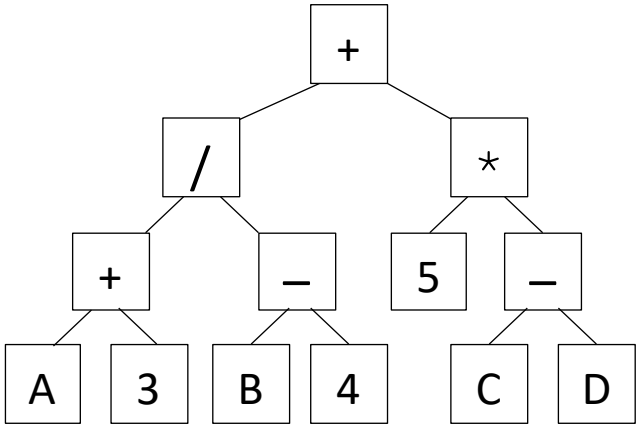
```
void inorder(Node *p) {
  if (p == nullptr) return;
  inorder(p->leftChild);
  visit(p);
  inorder(p->rightChild);
}
```



Infix Notation: <u>D B H E</u> A <u>F J C G</u>

Inorder Traversal

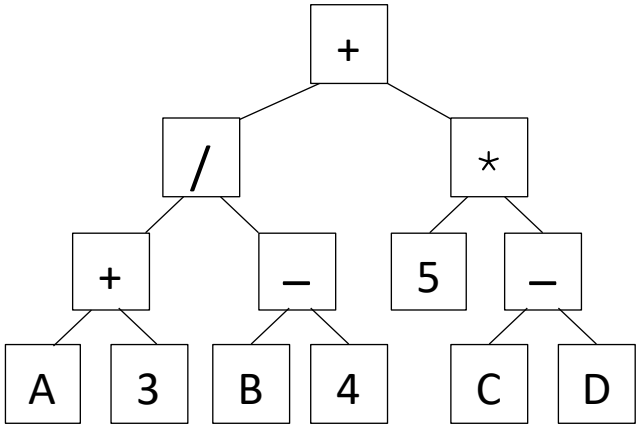
```
void inorder(Node *p) {
  if (p == nullptr) return;
  inorder(p->leftChild);
  visit(p);
  inorder(p->rightChild);
}
```



Infix: try it

Inorder Traversal

```
void inorder(Node *p) {
  if (p == nullptr) return;
  inorder(p->leftChild);
  visit(p);
  inorder(p->rightChild);
}
```



Infix: <u>A + 3 / B – 4</u> + <u>5 * C – D</u>

Parenthesized infix: ((A + 3)/(B – 4)) + (5 * (C – D))

Binary Tree Traversals

Breadth-First:

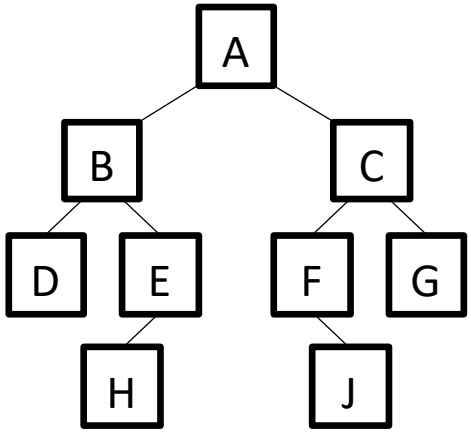
- Level-order

Level-order Traversal

```
void levelOrder(BinaryTree T) {
  Queue Q();       // FIFO
  Q.enqueue(T.root);
  while (not Q.isEmpty()) {
    Node *p = Q.dequeue();
    visit(p);
    if (p->leftChild != null)
      Q.enqueue(p->leftChild);
    if (p->rightChild != null)
      Q.enqueue(p->rightChild);
  }
}
```

Level-order Traversal

```
Q.enqueue(T.root);
while (not Q.isEmpty()) {
  Node *p = Q.dequeue();
  visit(p);
  if (p->leftChild != null) Q.enqueue(p->leftChild);
  if (p->rightChild != null) Q.enqueue(p->rightChild);
}
```



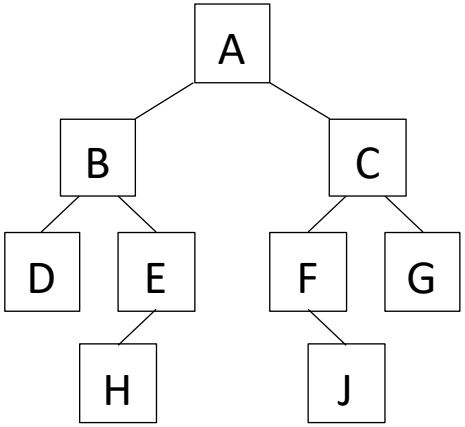
Queue

A
B
C
D
E
F
G
H
J

Level-order: A B C D E F G H J

Level-order Traversal

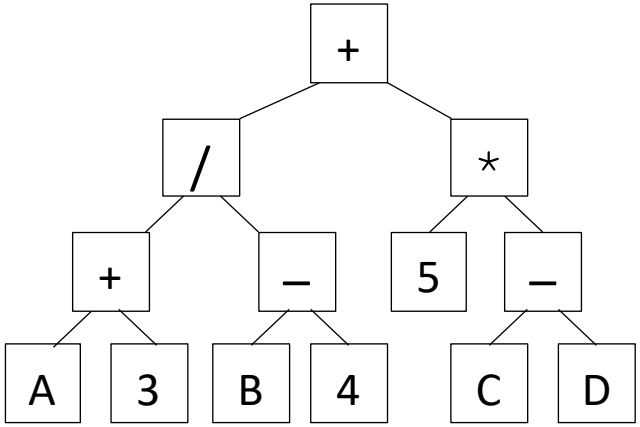
```
Q.enqueue(T.root);
while (not Q.isEmpty()) {
  Node *p = Q.dequeue();
  visit(p);
  if (p->leftChild != null) Q.enqueue(p->leftChild);
  if (p->rightChild != null) Q.enqueue(p->rightChild);
}
```



Level-order: A <u>B C</u> <u>D E F G</u> <u>H J</u>

Level-order Traversal

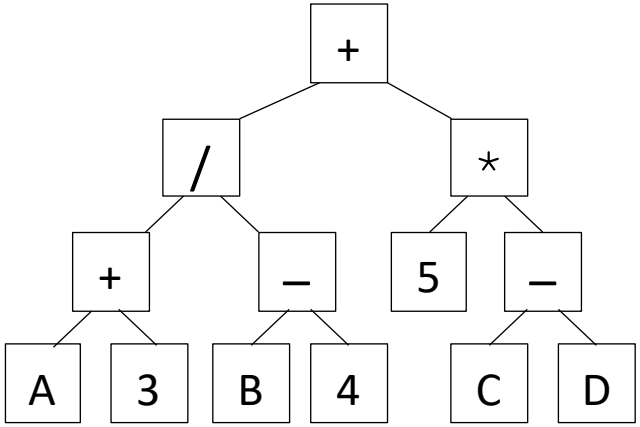
```
Q.enqueue(T.root);
while (not Q.isEmpty()) {
  Node *p = Q.dequeue();
  visit(p);
  if (p->leftChild != null) Q.enqueue(p->leftChild);
  if (p->rightChild != null) Q.enqueue(p->rightChild);
}
```



Level-order: try it

Level-order Traversal

```
Q.enqueue(T.root);
while (not Q.isEmpty()) {
  Node *p = Q.dequeue();
  visit(p);
  if (p->leftChild != null) Q.enqueue(p->leftChild);
  if (p->rightChild != null) Q.enqueue(p->rightChild);
}
```



Level-order: + <u>/ *</u> <u>+ – 5 –</u> <u>A 3 B 4 C D</u>

Analysis

- Let n = number of nodes in the tree.
- Each traversal spends $\theta(1)$ time at each node, so each traversal has $\theta(n)$ total running time.

A Postfix Expression

Postfix Expression:

An expression is called the postfix expression if the operator appears in the expression after the operands.

Example:

infix: A+B * (C-D) = AB+CD-*

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: 3 4 + 9 1 – *

op = ?

Stack:

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3~~ 4 + 9 1 – *

op = 3

Stack:

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3~~ 4 + 9 1 – *

op = 3

Stack: 3

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4~~ + 9 1 – *

op = 4

Stack: 3

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4~~ + 9 1 – *

op = 4

Stack: 3 4

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 +~~ 9 1 – *

op = +

Stack: 3 4

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 +~~ 9 1 – *

op = +

Stack: 7

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 + 9~~ 1 – *

op = 9

Stack: 7

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 + 9~~ 1 – *

op = 9

Stack: 7 9

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 + 9 1~~ – *

op = 1

Stack: 7 9

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 + 9 1~~ – *

op = 1

Stack: 7 9 1

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 + 9 1 –~~ *

op = –

Stack: 7 9 1

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 + 9 1 –~~ *

op = –

Stack: 7 8

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 + 9 1 – *~~

op = *

Stack: 7 8

Evaluating a Postfix Expression

```
evaluatePostfix() { // can use stack or recursion
  Stack S();
  while (op = read()) {
    if (op is operand)
      S.push(op);
    else if (op is operator) {
      right = S.pop();
      left = S.pop();
      S.push(apply(op, left, right));
    }
  }
  return S.pop();
}
```

Example: ~~3 4 + 9 1 – *~~

op = *

Stack: 56

A Prefix Expression

Prefix Expression:

An expression is called the prefix expression if the operator appears in the expression before the operands.

Example:

infix: (A+B) * (C-D) = *+AB-CD

Evaluating a Prefix Expression

```
evaluatePrefix() { // can use stack or recursion
  op = read();
  if (op is operand)
    return op;
  else if (op is operator) {
    left = evaluatePrefix();
    right = evaluatePrefix();
    return apply(op, left, right);
  }
}
```

Example: * + 3 4 – 9 1

op = ?

Action: ?

Evaluating a Prefix Expression

```
evaluatePrefix() { // can use stack or recursion
  op = read();
  if (op is operand)
    return op;
  else if (op is operator) {
    left = evaluatePrefix();
    right = evaluatePrefix();
    return apply(op, left, right);
  }
}
```

Example: * + 3 4 – 9 1

op = *

Action: ?

Evaluating a Prefix Expression

```
evaluatePrefix() { // can use stack or recursion
  op = read();
  if (op is operand)
    return op;
  else if (op is operator) {
    left = evaluatePrefix();
    right = evaluatePrefix();
    return apply(op, left, right);
  }
}
```

Example: ~~* + 3 4~~ – 9 1

op = *

Action:
Recursively read and evaluate + 3 4 $\Rightarrow$ left = 7

Evaluating a Prefix Expression

```
evaluatePrefix() { // can use stack or recursion
  op = read();
  if (op is operand)
    return op;
  else if (op is operator) {
    left = evaluatePrefix();
    right = evaluatePrefix();
    return apply(op, left, right);
  }
}
```

Example: ~~* + 3 4 – 9 1~~

op = *

Action:
Recursively read and evaluate + 3 4 $\Rightarrow$ left = 7
Recursively read and evaluate – 9 1 $\Rightarrow$ right = 8

Evaluating a Prefix Expression

```
evaluatePrefix() { // can use stack or recursion
  op = read();
  if (op is operand)
    return op;
  else if (op is operator) {
    left = evaluatePrefix();
    right = evaluatePrefix();
    return apply(op, left, right);
  }
}
```

Example: ~~* + 3 4 – 9 1~~

op = *

Action:
Recursively read and evaluate + 3 4 $\Rightarrow$ left = 7
Recursively read and evaluate – 9 1 $\Rightarrow$ right = 8
apply(*, 7, 8) $\Rightarrow$ return 56

Evaluating a Prefix Expression

```
evaluatePrefix("* + 3 4 - 9 1")
│
├── op = "*" (operator)
│   ├── left = evaluatePrefix("+ 3 4")
│   │   ├── op = "+" (operator)
│   │   │   ├── left = evaluatePrefix("3")
│   │   │   │   └── op = 3 (operand) → return 3
│   │   │   └── right = evaluatePrefix("4")
│   │   │       └── op = 4 (operand) → return 4
│   │   └── apply(+, 3, 4) → return 7
│   │
│   └── left subtree result = 7
│
└── right = evaluatePrefix("- 9 1")
    ├── op = "-" (operator)
    │   ├── left = evaluatePrefix("9")
    │   │   └── op = 9 (operand) → return 9
    │   └── right = evaluatePrefix("1")
    │       └── op = 1 (operand) → return 1
    └── apply(-, 9, 1) → return 8
    │
    └── right subtree result = 8

Final: apply(*, 7, 8) → 56
```

Questions?